



يونيسف سلطان زين العابدين  
**unisza**  
UNIVERSITI SULTAN ZAINAL ABIDIN

**SEMESTER : 5**

**SESSION : SESSION 1 2025/20264**

**CODE : CSA34003**

**COURSE : PENETRATION TESTING**

**SUPERVISOR : AHMAD FAISAL AMRI BIN ABIDIN @ BHARUN**

**TITLE : ASSIGNMENT 1: NETWORK TRAFFIC CAPTURE AND  
ANALYSIS USING WIRESHARK**

| NAME                                 | MATRICS NUMBER |
|--------------------------------------|----------------|
| ALIAH THAQIFAH BINTI KHAIRUL ARIFFIN | 079205         |

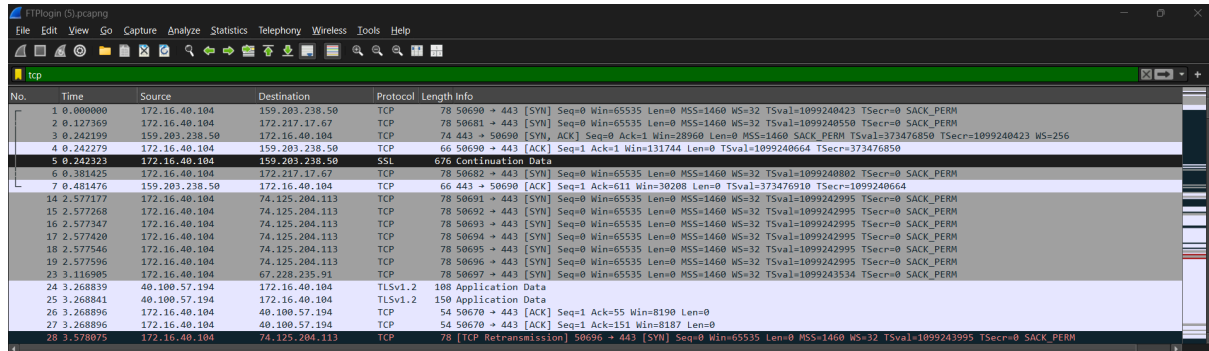
## TABLE OF CONTENTS

|                                                                         |          |
|-------------------------------------------------------------------------|----------|
| <b>PART 1: INSPECTING PROTOCOL LAYERS.....</b>                          | <b>2</b> |
| 1.1 TCP Handshake in the Packet List Pane.....                          | 2        |
| 1.2 The OSI Layer 1–4 Information Shown in the Packet Details Pane..... | 2        |
| 1.3 The Raw Data Displayed in the Packet Bytes Pane.....                | 3        |
| <b>Part 2: Extracting Credentials from Unencrypted Protocols.....</b>   | <b>4</b> |
| A. FTP Credentials.....                                                 | 4        |
| B. HTTP Login Credentials.....                                          | 5        |
| C. Identifying Successful Login Attempt.....                            | 6        |
| <b>Part 3: HTTP Basic Authentication Analysis.....</b>                  | <b>7</b> |

## PART 1: INSPECTING PROTOCOL LAYERS

### 1.1 TCP Handshake in the Packet List Pane

1. Apply TCP filter by typing “ftp” and click on “Enter”.
2. The TCP three-way handshake consists of a SYN from the client which requests a TCP connection, a SYN-ACK reply from the server and a final ACK from the client.
3. This process synchronizes sequence numbers and confirms that both sides are ready. After that, a TCP connection is established before any data is transmitted.



| No. | Time     | Source         | Destination    | Protocol | Length | Info                                                                                                           |
|-----|----------|----------------|----------------|----------|--------|----------------------------------------------------------------------------------------------------------------|
| 1   | 0.000000 | 172.16.40.104  | 159.203.238.50 | TCP      | 78     | 50690 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099240423 TSecr=0 SACK_PERM                      |
| 2   | 0.127369 | 172.16.40.104  | 172.217.17.67  | TCP      | 78     | 50681 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099240550 TSecr=0 SACK_PERM                      |
| 3   | 0.242199 | 159.203.238.50 | 172.16.40.104  | TCP      | 74     | 443 → 50690 [SYN, ACK] Seq=1 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=373476850 TSecr=1099240423 WS=256  |
| 4   | 0.242279 | 172.16.40.104  | 159.203.238.50 | TCP      | 66     | 50690 → 443 [ACK] Seq=1 Ack=1 Win=131744 Len=0 TSval=1099240664 TSecr=373476850                                |
| 5   | 0.242223 | 172.16.40.104  | 159.203.238.50 | TCP      | 78     | 50682 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099240802 TSecr=0 SACK_PERM                      |
| 6   | 0.381425 | 172.16.40.104  | 172.217.17.67  | TCP      | 66     | 443 → 50690 [ACK] Seq=1 Ack=611 Win=30208 Len=0 TSval=373476910 TSecr=1099240664                               |
| 7   | 0.481476 | 159.203.238.50 | 172.16.40.104  | TCP      | 78     | 50691 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 14  | 2.577177 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50692 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 15  | 2.577268 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50693 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 16  | 2.577347 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50694 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 17  | 2.577420 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50695 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 18  | 2.577546 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50696 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 19  | 2.577596 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | 50697 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099242995 TSecr=0 SACK_PERM                      |
| 23  | 3.116995 | 172.16.40.104  | 67.228.235.91  | TCP      | 78     | 50697 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099243534 TSecr=0 SACK_PERM                      |
| 24  | 3.268839 | 40.100.57.194  | 172.16.40.104  | TLSv1.2  | 108    | Application Data                                                                                               |
| 25  | 3.268841 | 40.100.57.194  | 172.16.40.104  | TLSv1.2  | 150    | Application Data                                                                                               |
| 26  | 3.268896 | 172.16.40.104  | 40.100.57.194  | TCP      | 54     | 50670 → 443 [ACK] Seq=1 Ack=55 Win=8190 Len=0                                                                  |
| 27  | 3.268896 | 172.16.40.104  | 40.100.57.194  | TCP      | 54     | 50670 → 443 [ACK] Seq=1 Ack=151 Win=8187 Len=0                                                                 |
| 28  | 3.578075 | 172.16.40.104  | 74.125.204.113 | TCP      | 78     | [TCP Retransmission] 50690 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=32 TSval=1099243995 TSecr=0 SACK_PERM |

### 1.2 The OSI Layer 1–4 Information Shown in the Packet Details Pane.

#### Layer 1 (Physical Layer)

The frame section shows information related to the physical transmission, such as bytes on wire, the size of the frame, timestamps and the interface that captured it.

```
Frame 169: 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0, id 0
  Section number: 1
    Interface id: 0 (en0)
    Encapsulation type: Ethernet (1)
    Arrival Time: May 11, 2018 11:45:28.838059000 Malay Peninsula Standard Time
    UTC Arrival Time: May 11, 2018 03:45:28.838059000 UTC
    Epoch Arrival Time: 1526010328.838059000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.000405000 seconds]
    [Time delta from previous displayed frame: 0.037511000 seconds]
    [Time since reference or first frame: 22.846911000 seconds]
    Frame Number: 169
    Frame Length: 78 bytes (624 bits)
    Capture Length: 78 bytes (624 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp]
    [Coloring Rule Name: TCP SYN/FIN]
    [Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]
```

#### Layer 2 (Data Link Layer)

The Ethernet II section shows MAC addresses for both destination and source and ether type.

```
Ethernet II, Src: Apple_c3:9e:b3 (ac:bc:32:c3:9e:b3), Dst: RuijieNetwor_35:38:33 (58:69:6c:35:38:33)
  Destination: RuijieNetwor_35:38:33 (58:69:6c:35:38:33)
  Source: Apple_c3:9e:b3 (ac:bc:32:c3:9e:b3)
  Type: IPv4 (0x0800)
  [Stream index: 0]
```

## Layer 3 (Network Layer)

The Internet Protocol panel shows IP addresses, Time-to-Live (TTL) and protocol field.

```
Internet Protocol Version 4, Src: 172.16.40.104, Dst: 107.152.24.197
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 64
    Identification: 0x9589 (38281)
  ▶ 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0x4c59 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 172.16.40.104
    Destination Address: 107.152.24.197
    [Stream index: 13]
```

## Layer 4 (Transport Layer)

The Transmission Control Protocol panel shows ports and TCP flags.

```
Transmission Control Protocol, Src Port: 50702, Dst Port: 443, Seq: 0, Len: 0
  Source Port: 50702
  Destination Port: 443
  [Stream index: 16]
  [Stream Packet Number: 1]
  ▶ [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1574302459
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1011 .... = Header Length: 44 bytes (11)
  ▶ Flags: 0x002 (SYN)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0x1703 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▶ Options: (24 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No
```

## 1.3 The Raw Data Displayed in the Packet Bytes Pane.

Shows actual raw binary/hex representing the packet.

```
0000 58 69 6c 35 38 33 ac bc 32 c3 9e b3 08 00 45 00 Xil583.. 2...E
0010 00 40 95 89 40 00 40 06 4c 59 ac 10 28 68 6b 98 .@..@.@ LY..(h
0020 18 c5 c6 0e 01 bb 5d d5 f2 fb 00 00 00 00 b0 02 .....].....
0030 ff ff 17 03 00 00 02 04 05 b4 01 03 03 05 01 01 .....
0040 08 0a 41 85 6d 04 00 00 00 00 04 02 00 00 ..A.m.....
```

## 1.4 Explanation of How Wireshark Displays Packet Structure

### 1. Packet List Pane (Top Pane)

- Displays one line per packet.
- Shows packet number, timestamp, source and destination addresses, protocol, length along with some information regarding the packet.

### 2. Packet Details Pane (Middle Pane)

- When a user selected a packet, the middle pane would display protocol layers that appear in several expendable sections like Frame, Ethernet II, Internet Protocol and Transmission Control Protocol (TCP).

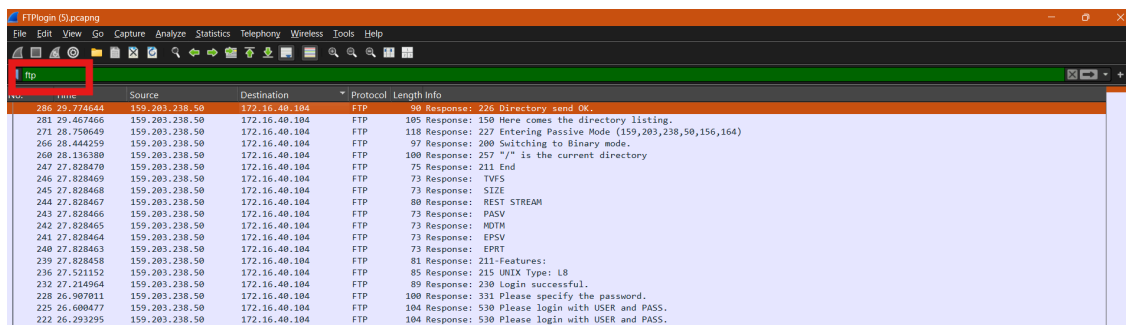
### 3. Packet Bytes Pane (Bottom Pane)

- The bottom pane shows the raw packet contents in both hexadecimal and ASCII form.
- The left side contains byte offsets, the middle shows hex values and the right side shows the ASCII characters.

## PART 2: EXTRACTING CREDENTIALS FROM UNENCRYPTED PROTOCOLS

### A. FTP Credentials

1. Apply an ftp display filter.



| No. | Time      | Source         | Destination   | Protocol | Length | Info                                                         |
|-----|-----------|----------------|---------------|----------|--------|--------------------------------------------------------------|
| 286 | 29.745644 | 159.203.238.50 | 172.16.40.104 | FTP      | 236    | Response: 226 Binary send OK.                                |
| 281 | 29.467466 | 159.203.238.50 | 172.16.40.104 | FTP      | 105    | Response: 150 Here comes the directory listing.              |
| 271 | 28.750649 | 159.203.238.50 | 172.16.40.104 | FTP      | 118    | Response: 227 Entering Passive Mode (159,203,238,50,156,164) |
| 266 | 28.444259 | 159.203.238.50 | 172.16.40.104 | FTP      | 97     | Response: 200 Switching to Binary mode.                      |
| 260 | 28.136380 | 159.203.238.50 | 172.16.40.104 | FTP      | 100    | Response: 257 "/" is the current directory                   |
| 247 | 27.828470 | 159.203.238.50 | 172.16.40.104 | FTP      | 75     | Response: 211 End                                            |
| 246 | 27.828469 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: TVFS                                               |
| 242 | 27.828468 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: SIZE                                               |
| 244 | 27.828467 | 159.203.238.50 | 172.16.40.104 | FTP      | 80     | Response: REST STREAM                                        |
| 243 | 27.828466 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: PASV                                               |
| 241 | 27.828465 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: MDTM                                               |
| 240 | 27.828463 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: EPSV                                               |
| 239 | 27.828458 | 159.203.238.50 | 172.16.40.104 | FTP      | 73     | Response: EPRT                                               |
| 236 | 27.521152 | 159.203.238.50 | 172.16.40.104 | FTP      | 81     | Response: 211-Features:                                      |
| 232 | 27.214964 | 159.203.238.50 | 172.16.40.104 | FTP      | 85     | Response: 215 UNIX Type: L8                                  |
| 228 | 26.907011 | 159.203.238.50 | 172.16.40.104 | FTP      | 89     | Response: 230 Login successful.                              |
| 225 | 26.600477 | 159.203.238.50 | 172.16.40.104 | FTP      | 100    | Response: 331 Please specify the password.                   |
| 222 | 26.293295 | 159.203.238.50 | 172.16.40.104 | FTP      | 104    | Response: 530 Please login with USER and PASS.               |
| 222 | 26.293295 | 159.203.238.50 | 172.16.40.104 | FTP      | 104    | Response: 530 Please login with USER and PASS.               |

2. Identify the username and password transmitted during login.

**Username:** john

**Password:** Flapper

| No. | Time      | Source         | Destination    | Protocol | Length | Info                                           |
|-----|-----------|----------------|----------------|----------|--------|------------------------------------------------|
| 212 | 25.404787 | 159.203.238.50 | 172.16.40.104  | FTP      | 89     | Response: 230 Login successful.                |
| 202 | 24.346566 | 159.203.238.50 | 172.16.40.104  | FTP      | 100    | Response: 331 Please specify the password.     |
| 188 | 23.278382 | 159.203.238.50 | 172.16.40.104  | FTP      | 104    | Response: 530 Please login with USER and PASS. |
| 175 | 23.045926 | 159.203.238.50 | 172.16.40.104  | FTP      | 104    | Response: 530 Please login with USER and PASS. |
| 161 | 21.842290 | 159.203.238.50 | 172.16.40.104  | FTP      | 86     | Response: 220 (vsFTPd 3.0.3)                   |
| 273 | 28.751511 | 172.16.40.104  | 159.203.238.50 | FTP      | 72     | Request: LIST                                  |
| 268 | 28.444571 | 172.16.40.104  | 159.203.238.50 | FTP      | 72     | Request: PASV                                  |
| 262 | 28.141781 | 172.16.40.104  | 159.203.238.50 | FTP      | 74     | Request: TYPE I                                |
| 257 | 27.829186 | 172.16.40.104  | 159.203.238.50 | FTP      | 71     | Request: PWD                                   |
| 238 | 27.521460 | 172.16.40.104  | 159.203.238.50 | FTP      | 72     | Request: FEAT                                  |
| 234 | 27.215261 | 172.16.40.104  | 159.203.238.50 | FTP      | 72     | Request: SYST                                  |
| 230 | 26.907334 | 172.16.40.104  | 159.203.238.50 | FTP      | 80     | Request: PASS Flapper                          |
| 227 | 26.600816 | 172.16.40.104  | 159.203.238.50 | FTP      | 77     | Request: USER john                             |
| 224 | 26.293588 | 172.16.40.104  | 159.203.238.50 | FTP      | 76     | Request: AUTH SSL                              |
| 220 | 25.986193 | 172.16.40.104  | 159.203.238.50 | FTP      | 76     | Request: AUTH TLS                              |
| 204 | 24.346737 | 172.16.40.104  | 159.203.238.50 | FTP      | 80     | Request: PASS Flapper                          |
| 190 | 23.278703 | 172.16.40.104  | 159.203.238.50 | FTP      | 77     | Request: USER john                             |
| 177 | 23.046193 | 172.16.40.104  | 159.203.238.50 | FTP      | 76     | Request: AUTH SSL                              |
| 163 | 21.842455 | 172.16.40.104  | 159.203.238.50 | FTP      | 76     | Request: AUTH TLS                              |

3. Explain why FTP is considered insecure based on your findings.

FTP is considered insecure because it transmits inputs like login credentials (usernames and passwords) over the network without encryption, as shown in Part 2(A)(2), making them appear in plaintext and thus easier to be seen and misused by unauthorized personnels.

## B. HTTP Login Credentials

1. Apply an http display filter.

| http |              |              |              |          |                                                                  |
|------|--------------|--------------|--------------|----------|------------------------------------------------------------------|
| No.  | Time         | Source       | Destination  | Protocol | Length Info                                                      |
| 20   | 9.633426226  | 172.16.1.1   | 172.16.1.129 | HTTP     | 504 GET /login.htm HTTP/1.1                                      |
| 22   | 9.634160517  | 172.16.1.129 | 172.16.1.1   | HTTP     | 674 HTTP/1.1 200 OK (text/html)                                  |
| 24   | 9.739398054  | 172.16.1.1   | 172.16.1.129 | HTTP     | 470 GET /favicon.ico HTTP/1.1                                    |
| 25   | 9.739677846  | 172.16.1.129 | 172.16.1.1   | HTTP     | 569 HTTP/1.1 404 Not Found (text/html)                           |
| 49   | 21.631365064 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 51   | 21.632384019 | 172.16.1.129 | 172.16.1.1   | HTTP     | 448 HTTP/1.1 200 OK (text/html)                                  |
| 66   | 27.780805408 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 68   | 27.781384254 | 172.16.1.129 | 172.16.1.1   | HTTP     | 447 HTTP/1.1 200 OK (text/html)                                  |
| 84   | 33.813057139 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 86   | 33.813631007 | 172.16.1.129 | 172.16.1.1   | HTTP     | 419 HTTP/1.1 200 OK (text/html)                                  |

2. Locating the first HTTP POST request.

| No. | Time         | Source       | Destination  | Protocol | Length Info                                                      |
|-----|--------------|--------------|--------------|----------|------------------------------------------------------------------|
| 86  | 33.813631007 | 172.16.1.129 | 172.16.1.1   | HTTP     | 419 HTTP/1.1 200 OK (text/html)                                  |
| 84  | 33.813057139 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 68  | 27.781384254 | 172.16.1.129 | 172.16.1.1   | HTTP     | 447 HTTP/1.1 200 OK (text/html)                                  |
| 66  | 27.780805408 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 51  | 21.632384019 | 172.16.1.129 | 172.16.1.1   | HTTP     | 448 HTTP/1.1 200 OK (text/html)                                  |
| 49  | 21.631365064 | 172.16.1.1   | 172.16.1.129 | HTTP     | 657 POST /login.php HTTP/1.1 (application/x-www-form-urlencoded) |
| 25  | 9.739677846  | 172.16.1.129 | 172.16.1.1   | HTTP     | 569 HTTP/1.1 404 Not Found (text/html)                           |
| 24  | 9.739398054  | 172.16.1.1   | 172.16.1.129 | HTTP     | 470 GET /favicon.ico HTTP/1.1                                    |
| 22  | 9.634160517  | 172.16.1.129 | 172.16.1.1   | HTTP     | 674 HTTP/1.1 200 OK (text/html)                                  |
| 20  | 9.633426226  | 172.16.1.1   | 172.16.1.129 | HTTP     | 504 GET /login.htm HTTP/1.1                                      |

3. Identify the username and password sent through the POST request and

```
File Data: 31 bytes
  HTML Form URL Encoded: application/x-www-form-urlencoded
    Form item: "u" = "Isaac"
    Form item: "p" = "Flapper"
    Form item: canvas = Submit"
```

Username ("u") = Isaac

Password ("p") = Flapper

#### 4. Follow the TCP stream and describe what information is or is not visible and why.

```

Wireshark - Follow TCP Stream (tcp.stream eq 1) - httpLogin (11).pcapng

GET /login.htm HTTP/1.1
Host: 172.16.1.129
Connection: keep-alive
Pragma: no-cache
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/67.0.3396.99 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

HTTP/1.1 200 OK
Date: Thu, 28 Jun 2018 18:57:10 GMT
Server: Apache/2.4.29 (Debian)
Last-Modified: Thu, 28 Jun 2018 18:50:11 GMT
ETag: "204-56fb833770e07-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 271
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html

.....QQn.....,~i+g..a...8..4...V...J...}.....o..1..}[0...y...1.F[.4.)..Kta..o6<..-1..Xpl..3..s...~..li....N%t...'.o)
1....f.*.../Q+...6...l.....p....*,e9uX;|@....e.)k...R.I.;|...yy.rv;T..Ix.....o.H.e.-...-(#\...5..A.]Z7{/.k.....
)W.....
GET /favicon.ico HTTP/1.1
Host: 172.16.1.129
Connection: keep-alive
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/67.0.3396.99 Safari/537.36
Accept: image/webp,image/apng,image/*,*/*;q=0.8
Referer: http://172.16.1.129/login.htm
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

Packet 20, 2 clicks pkts, 2 covers pkts, 3 turns. Click to select.
Entire conversation (1953 bytes) Show as ASCII No delta times Stream 1

```

Wireshark displays HTTP request and response headers in readable text since headers are never compressed and must be in plain ASCII format for browsers and servers to interpret. The unreadable section is the webpage body. It appears encrypted-like due to it being compressed using gzip (“Content-Encoding: gzip”) and Wireshark does not automatically decompress gzip data, so the binary compressed HTML appears as unreadable characters.

### C. Identifying Successful Login Attempt

1. Review subsequent packets to determine which password attempt was accepted.
2. Identify the correct password and explain how you determined it using packet responses.

#### 1st Attempt (Packet 49, 51): Failed

```

HTML Form URL Encoded: application/x-www-form-urlencoded
  ▶ Form item: "u" = "Isaac"
  ▶ Form item: "p" = "Flapper"
  ▶ Form item: "canvas" = "Submit"

▶ Frame 51: 448 bytes on wire (3584 bits), 448 bytes captured (3584 bits) on interface eth0, id 0
▶ Ethernet II, Src: VMware_e1:fe:3f (00:0c:29:e1:fe:3f), Dst: VMware_c0:00:08 (00:50:56:c0:00:08)
▶ Internet Protocol Version 4, Src: 172.16.1.129, Dst: 172.16.1.1
▶ Transmission Control Protocol, Src Port: 80, Dst Port: 55694, Seq: 1, Ack: 592, Len: 382
▶ Hypertext Transfer Protocol
  ▶ Line-based text data: text/html (1 lines)
    You entered: <b>Isaac</b> and <b>Flapper</b><p><h1>Login Denied!</h1><h3>Click your browser's back button to try a

```

## 2nd Attempt (Packet 66, 68) : Failed

```
HTML Form URL Encoded: application/x-www-form-urlencoded
  ▶ Form item: "u" = "Isaac"
  ▶ Form item: "p" = "Snapper"
  ▶ Form item: "canvas" = "Submit"

Line-based text data: text/html (1 lines)
  You entered: <b>Isaac</b> and <b>Snapper<p><h1>Login Denied!</h1><h3>Click your br
```

## 3rd Attempt (Packet 84, 86): Successful

```
HTML Form URL Encoded: application/x-www-form-urlencoded
  ▶ Form item: "u" = "Isaac"
  ▶ Form item: "p" = "Slapper"
  ▶ Form item: "canvas" = "Submit"

Hypertext Transfer Protocol
Line-based text data: text/html (1 lines)
  You entered: <b>Isaac</b> and <b>Slapper<p><h1>Login Approved!</h1><h2>Welcome, Isaac</h1>\n
```

Password attempt was accepted on the 3rd try (Packet 84 and 86) with the right password being “Slapper”. The correct password is determined by looking at the Line-based text data section where it shows texts like “Login Denied” or “Login Approved”. On the 3rd attempt, we can see the latter along with “Welcome Isaac”, which signify that his 3rd attempt was a success.

## PART 3: HTTP BASIC AUTHENTICATION ANALYSIS

1. Apply a http filter to locate packets. Packet 17 displaying texts on Unauthorized access that might have been attempted, yet no sign of credentials.

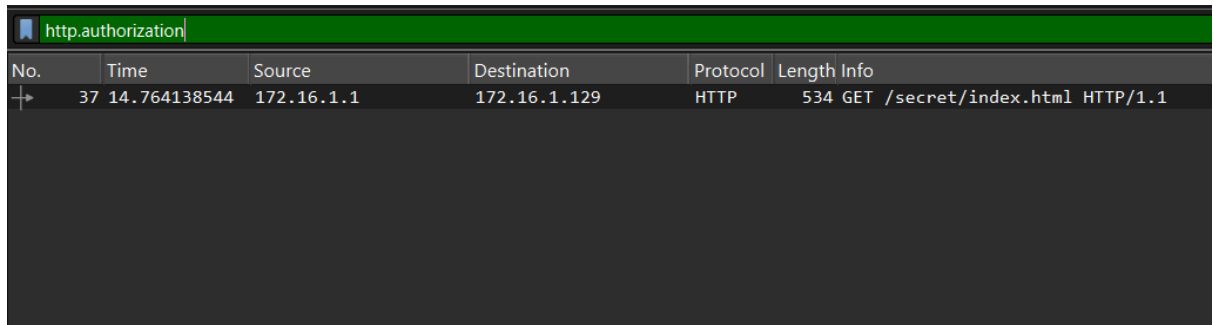
The image shows a Wireshark packet capture with a filter set to 'http'. The packet list shows several HTTP requests and responses. Packet 17 is highlighted, showing an HTTP 401 Unauthorized response. The packet details pane shows the response body, which is an HTML page indicating that the user is not authorized to access the document.

| No. | Time         | Source       | Destination  | Protocol | Length | Info                                  |
|-----|--------------|--------------|--------------|----------|--------|---------------------------------------|
| 15  | 4.763321310  | 172.16.1.1   | 172.16.1.129 | HTTP     | 487    | GET /secret/index.html HTTP/1.1       |
| 17  | 4.763885968  | 172.16.1.129 | 172.16.1.1   | HTTP     | 811    | HTTP/1.1 401 Unauthorized (text/html) |
| 37  | 14.764138544 | 172.16.1.1   | 172.16.1.129 | HTTP     | 534    | GET /secret/index.html HTTP/1.1       |
| 39  | 14.765466565 | 172.16.1.129 | 172.16.1.1   | HTTP     | 514    | HTTP/1.1 200 OK (text/html)           |
| 42  | 15.489831529 | 172.16.1.1   | 172.16.1.129 | HTTP     | 453    | GET /favicon.ico HTTP/1.1             |
| 43  | 15.490024569 | 172.16.1.129 | 172.16.1.1   | HTTP     | 569    | HTTP/1.1 404 Not Found (text/html)    |

```
\r\n
[Request in frame: 15]
[Time since request: 0.000564658 seconds]
[Request URI: /secret/index.html]
[Full request URI: http://172.16.1.129/secret/index.html]
File Data: 459 bytes
Line-based text data: text/html (14 lines)
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">\n
<html><head>\n
<title>401 Unauthorized</title>\n
</head><body>\n
<h1>Unauthorized</h1>\n
<p>This server could not verify that you\n
are authorized to access the document\n
requested. Either you supplied the wrong\n
credentials (e.g., bad password), or your\n
browser doesn't understand how to supply\n
the credentials required.</p>\n
</body>\n
</html>
```



2. Apply a http.authorization filter to locate authenticated packets as this filter only shows HTTP packets that contain an Authorization header.

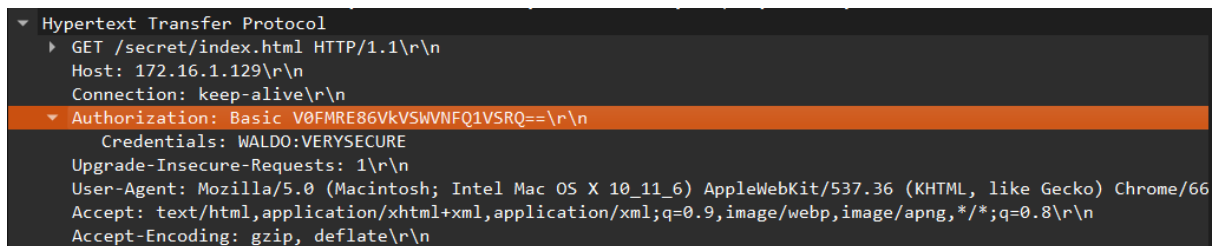


| No. | Time         | Source     | Destination  | Protocol | Length | Info                            |
|-----|--------------|------------|--------------|----------|--------|---------------------------------|
| 37  | 14.764138544 | 172.16.1.1 | 172.16.1.129 | HTTP     | 534    | GET /secret/index.html HTTP/1.1 |

3. Click on Packet 37 > expand Hypertext Transfer Protocol > expand Authorization.

Username: WALDO

Password: VERYSECURE



```
▼ Hypertext Transfer Protocol
  ► GET /secret/index.html HTTP/1.1\r\n
    Host: 172.16.1.129\r\n
    Connection: keep-alive\r\n
  ▼ Authorization: Basic V0FMRE86VkVSWVNFQ1VSRQ==\r\n
    Credentials: WALDO:VERYSECURE
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/66
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8\r\n
  Accept-Encoding: gzip, deflate\r\n
```

4. How Wireshark decodes the Base64-encoded credentials and why is this method weak?

- HTTP Basic Authentication sends credentials in the form “username:password” encoded using Base64. This is reversible encoding and not encryption.
- The Base64 string inside the “Authorization: Basic” header is detected automatically by Wireshark and decoded into readable text.
- Base64 is not a security mechanism, so those who capture the traffic can decode the credentials thus making Basic Authentication weak and unsuitable for unencrypted HTTP connections.
- An attacker who captures the traffic can decode the string and recover the username and password immediately. Without HTTPS, the authentication is not protected.